CESI
ÉCOLE D'INGÉNIEURS

Project Modeling and Databases

Group 1B

Delivrable 2

ARNALOT-BY Maëlle ; RAS Mohamed ; LETT Killian ; PHILIPPE Mathéo

Date : May 15, 2025

CESI
ÉCOLE D'INGÉNIEURS

Summary :

INTRODUCTION : 3

I- SQL requests : 4

Query 1 : List of all agencies 4

Query 2 : List of all technical staff members from the Bordeaux agency 4

Query 3 : Total number of deployed sensors* 5

Query 4 : List of reports published between 2018 and 2022 5

Query 5 : Display CH4 concentrations (in ppm) in the regions "Île-de-France", "Brittany", and "Occitanie" for May and June 2023 6

Query 6 : List of names of technical staff maintaining sensors related to industrial greenhouse gases 8

Query 7 : Titles and dates of reports concerning NH3 concentrations, sorted in reverse chronological order 9

Query 8 : Display the month with the lowest PFC concentration for each region 10

Query 9 : Average concentrations (in ppm) in the "Île-de-France" region in 2020, for each studied gas 11

Query 10 : Productivity rate of administrative staff from the Toulouse agency 12

Query 11 : For a given gas, list of reports containing data related to it 12

Query 12 : List of regions where the number of sensors exceeds the number of agency staff 13

User Accounts and Access Rights Management : 14

II- Database generation and populating: 15

1 – Overview of Provided Data : 15

2 – Methodology of Data Populating : 15

3 – Applied Constraints and Data Integrity : 16

4 – Challenges and Solutions : 16

III- Comparison between implemented and expected functionalities : 18

1 – Summary Table : 18

2 – Global Assessment : 19

IV- Evolution from Delivrable 1 : 20

1 – Conceptual Data Model : 20

2 – Logical Data Model : 21

3 – Physical Data Model : 22

V- CONCLUSION : 24

VI- Technical Appendices : 24

CESI
ÉCOLE D'INGÉNIEURS

INTRODUCTION :

As part of the national "Air Quality" initiative led by the French Ministry of Ecological Transition and entrusted to the company ClearData, our team is now advancing to the implementation phase of a centralized information system designed to collect, store, and query environmental air quality data across metropolitan France. This project addresses the challenges raised in the latest IPCC report and supports France's broader efforts toward ecological transition.

In Deliverable 1, we focused on designing the database model, following best practices in relational modeling. This included a comprehensive data dictionary, a conceptual data model in third normal form (3NF), its logical and physical implementations, and relational algebra trees for the key queries specified by the client. This modeling phase enabled us to structure the client's needs into a robust and scalable data model.

Deliverable 2 marks a significant milestone in the project: the implementation of the database in a MySQL environment, the population of the system with realistic and coherent data, and the implementation and validation of functional SQL queries based on client requirements. This deliverable demonstrates the technical feasibility of the system through:

- The creation of the database schema and user accounts with appropriate permissions
- The insertion of structured data related to sensors, staff, measurements, and reports
- The execution of all client-requested queries using SQL
- A detailed comparison between implemented functionalities and initial project specifications

This deliverable serves as a proof of concept, showcasing the transition from theoretical design to a fully operational relational database, ready for demonstration and further development.

CESI
ÉCOLE D'INGÉNIEURS

I- SQL requests :

Query 1 : List of all agencies

- **SQL code :**

```sql
-- 1. Liste de l'ensemble des agences
SELECT * FROM agency;
```

- **Explanation :** Selects all columns (*) from all rows in the agency table.

- **Output :**

ID_Agency	Address	city_ID_City
1	27 Boulevard de la Mer, Ajaccio, 20000	1
2	89 Quai du Vin, Bordeaux, 33000	9
3	45 Rue des Vignes, Dijon, 21000	17
4	62 Avenue des Flandres, Lille, 59000	23
5	30 Place du Rhône, Lyon, 69000	25
6	74 Boulevard du Vieux-Port, Marseille, 13...	26
7	21 Allée de l'Estuaire, Nantes, 44000	31
8	53 Rue de la Loire, Orléans, 45000	35
9	99 Boulevard Lumière, Paris, 75008	36
10	36 Rue des Horizons, Rennes, 35000	42
11	18 Quai de Seine, Rouen, 76000	43
12	50 Avenue du Rhin, Strasbourg, 67000	45
13	12 Rue des Étoiles, Toulouse, 31000	47
NULL	NULL	NULL

Query 2 : List of all technical staff members from the Bordeaux agency

- **SQL code :**

```sql
-- 2. Liste du personnel technique de l'agence de Bordeaux
SELECT ID_Staff, First_Name, Last_Name, staff.Address, agency_ID_Agency, agency.Address, role.Role_Name FROM staff
INNER JOIN agency ON staff.agency_ID_Agency = agency.ID_Agency
INNER JOIN role ON staff.role_ID_Role = role.ID_Role
WHERE agency.Address LIKE '%Bordeaux%'
  AND role.ID_Role = 2;
```

- **Explanation :**

- SELECT : chooses the columns to display — in this case, staff information (ID_Staff, First_Name, Last_Name, Address), the agency's ID and address, and the role name (Role_Name).
- FROM staff : indicates that the query starts from the staff table, which contains employee data.

- INNER JOIN agency ON staff.agency_ID_Agency = agency.ID_Agency : joins the staff table with the agency table. An INNER JOIN means only rows with matching values in both tables are kept.
- INNER JOIN role ON staff.role_ID_Role = role.ID_Role : connects each employee to their role, only keeping those with a defined role.
- WHERE agency.Address LIKE '%Bordeaux%' : filters the results to keep only agencies whose address contains "Bordeaux".
- AND role.ID_Role = 2 : further filters the results to include only employees whose role ID is 2, which likely corresponds to technical staff.

- **Output :**

ID_Staff	First_Name	Last_Name	Address	agency_ID_Agency	Address	Role_Name
13	Alice	Dupont	45 Avenue Victor Hugo, Bordeaux, 33000	2	89 Quai du Vin, Bordeaux, 33000	technical agent
41	Morgan	Lambert	64 Boulevard Carnot, Limoges, 87000	2	89 Quai du Vin, Bordeaux, 33000	technical agent
49	Paul	Lecomte	34 Place de la Victoire, Poitiers, 86000	2	89 Quai du Vin, Bordeaux, 33000	technical agent
86	Maël	Prévost	19 Place des Arts, Bordeaux, 33000	2	89 Quai du Vin, Bordeaux, 33000	technical agent
99	Vincent	Gaudin	94 Boulevard de la Colline, Bordeaux, 33000	2	89 Quai du Vin, Bordeaux, 33000	technical agent

Query 3 : Total number of deployed sensors*

- **SQL code :**

```sql
-- 3. Nombre total de capteurs déployés
SELECT COUNT(*) AS total_sensors
FROM sensor;
```

- **Explanation :**

- SELECT COUNT(*) AS total_sensors: This part counts all the rows in the sensor table.
- COUNT(*) is a function that returns the total number of entries, and the result is labeled as total_sensors.
- FROM sensor: Indicates that the data is being counted from the sensor table.

- **Output :**

total_sensors
294

Query 4 : List of reports published between 2018 and 2022

- **SQL code :**

```sql
-- 4. Liste des rapports publiés entre 2018 et 2022
SELECT * FROM report
WHERE Date_Report BETWEEN '2018-01-01' AND '2022-12-31';
```

- **Explanation :**

- SELECT * FROM report: Retrieves all columns (*) from the report table, which contains report data.
- WHERE Date_Report BETWEEN '2018-01-01' AND '2022-12-31': Filters the results to include only the reports whose Date_Report falls between January 1st, 2018 and December 31st, 2022 (inclusive).
- The BETWEEN keyword is used to check if a value lies within a specific range.

- **Output :**

ID_Report	Title	Date_Report	staff_ID_Staff
9	Annual Report on N2O, O3 Concentration...	2019-05-03	68
10	Annual Report on O3, N2O, CO2 Concent...	2019-02-17	68
14	Annual Report on N2O, O3 Concentration...	2018-08-25	23
15	Annual Report on O3, HFC, N2O Concent...	2021-03-16	74
16	Annual Report on N2O, SF6, CH4 Concen...	2020-04-13	4
18	Annual Report on SF6 Concentration Mea...	2021-01-05	4
19	Annual Report on N2O, HFC, CH4 Concen...	2020-05-13	14
20	Annual Report on O3, CH4 Concentration...	2018-12-19	60
22	Annual Report on CO2, N2O, SF6 Concen...	2021-06-26	39
23	Annual Report on SF6, CO2, O3 Concentr...	2022-04-06	47
24	Annual Report on CO2, SF6 Concentratio...	2021-04-27	80
25	Annual Report on HFC, CO2, SF6 Concent...	2018-11-21	66
26	Annual Report on CO2, O3, HFC Concentr...	2018-08-25	80
27	Annual Report on CH4, SF6, O3 Concentr...	2019-11-01	51
29	Annual Report on CH4 Concentration Mea...	2021-10-16	1
31	Annual Report on SF6, HFC, O3 Concentr...	2022-06-06	80
36	Annual Report on N2O Concentration Me...	2018-09-21	66

Query 5 : Display CH4 concentrations (in ppm) in the regions "Île-de-France", "Brittany", and "Occitanie" for May and June 2023

- **SQL code :**

```sql
-- 5. Concentrations de CH4 en Ile-de-France, Bretagne, Occitanie en mai-juin 2023
SELECT region.Region_Name, city.City_Name, measure.Date_Measure, measure.Concentration FROM measure
JOIN sensor ON measure.Sensor_ID_Sensor = sensor.ID_Sensor
JOIN gas ON sensor.gas_ID_Gas = gas.ID_Gas
JOIN city ON sensor.city_ID_City = city.ID_City
JOIN region ON city.region_ID_Region = region.ID_Region
WHERE gas.Formula = 'CH4'
  AND region.Region_Name IN ('Ile-de-France', 'Bretagne', 'Occitanie')
  AND measure.Date_Measure BETWEEN '2023-05-01' AND '2023-06-30';
```

- **Explanation :**

- SELECT region.Region_Name, city.City_Name, measure.Date_Measure, measure.Concentration : Selects the region name, city name, date of measurement, and concentration value from the database.
- FROM measure: Starts from the measure table, which contains all recorded gas measurements.

- JOIN sensor ON measure.Sensor_ID_Sensor = sensor.ID_Sensor : Links each measurement to the corresponding sensor.
- JOIN gas ON sensor.gas_ID_Gas = gas.ID_Gas : Links each sensor to the type of gas it measures.
- JOIN city ON sensor.city_ID_City = city.ID_City : Connects the sensor to its city location.
- JOIN region ON city.region_ID_Region = region.ID_Region: Connects each city to its corresponding region.
- WHERE gas.Formula = 'CH4': Filters the results to include only measurements of methane (CH_4).
- AND region.Region_Name IN ('Île-de-France', 'Bretagne', 'Occitanie') : Limits results to those three specific French regions.
- AND measure.Date_Measure BETWEEN '2023-05-01' AND '2023-06-30' : Restricts results to measurements taken during May and June 2023.

- **Output :**

Region_Name	City_Name	Date_Measure	Concentration
Bretagne	Brest	2023-05-01	323.43
Bretagne	Brest	2023-06-01	427.03
Bretagne	Quimper	2023-05-01	281.11
Bretagne	Quimper	2023-06-01	235.34
Bretagne	Rennes	2023-05-01	131.49
Bretagne	Rennes	2023-06-01	213.70
Bretagne	Vannes	2023-05-01	365.88
Bretagne	Vannes	2023-06-01	210.49
Île-de-France	Créteil	2023-05-01	300.01
Île-de-France	Créteil	2023-06-01	85.08
Île-de-France	Nanterre	2023-05-01	140.60
Île-de-France	Nanterre	2023-06-01	365.72
Île-de-France	Paris	2023-05-01	65.71
Île-de-France	Paris	2023-06-01	358.81
Île-de-France	Versailles	2023-05-01	469.16
Île-de-France	Versailles	2023-06-01	234.89
Occitanie	Montpellier	2023-05-01	324.96
Occitanie	Montpellier	2023-06-01	90.83
Occitanie	Nîmes	2023-05-01	342.40
Occitanie	Nîmes	2023-06-01	188.55
Occitanie	Perpignan	2023-05-01	332.29
Occitanie	Perpignan	2023-06-01	282.58
Occitanie	Toulouse	2023-05-01	304.29
Occitanie	Toulouse	2023-06-01	420.98

CESI
ÉCOLE D'INGÉNIEURS

Query 6 : List of names of technical staff maintaining sensors related to industrial greenhouse gases

- **SQL code :**

```
-- 6. Noms des agents techniques liés aux GESI
SELECT DISTINCT staff.ID_Staff, staff.First_Name, staff.Last_Name, role.Role_Name FROM staff
JOIN sensor ON staff.ID_Staff = sensor.Staff_ID_Staff
JOIN gas ON sensor.gas_ID_Gas = gas.ID_Gas
JOIN type_gas ON gas.type_gas_ID_Gas = type_gas.ID_Gas
JOIN role ON staff.role_ID_Role = role.ID_Role
WHERE type_gas.ID_Gas = 2;
```

- **Explanation :**

- **SELECT DISTINCT staff.ID_Staff, staff.First_Name, staff.Last_Name, role.Role_Name**: Selects unique combinations of staff ID, first name, last name, and role name, avoiding duplicates thanks to DISTINCT.
- **FROM staff**: Starts from the staff table, which contains information about employees.
- **JOIN sensor ON staff.ID_Staff = sensor.Staff_ID_Staff**: Links each staff member to the sensors they are responsible for.
- **JOIN gas ON sensor.gas_ID_Gas = gas.ID_Gas**: Connects each sensor to the gas it measures.
- **JOIN type_gas ON gas.type_gas_ID_Gas = type_gas.ID_Gas**: Associates each gas with its type — here, we are targeting GESI.
- **JOIN role ON staff.role_ID_Role = role.ID_Role**: Retrieves the role (e.g., technician) of each staff member.
- **WHERE type_gas.ID_Gas = 2**: Filters the data to only include staff linked to **type of gas ID 2**, which corresponds to **GESI**.

- **Output :**

ID_Staff	First_Name	Last_Name	Role_Name
58	Sarah	Lévy	technical agent
77	Héloïse	Bisset	technical agent
87	Margaux	Lefort	technical agent
70	Corentin	Lacoste	technical agent
92	Océane	Dupré	technical agent
45	Noémie	Fontaine	technical agent
81	Justine	Roger	technical agent
63	Amélie	Poulain	technical agent
30	Louis	Guérin	technical agent
42	Nathan	Gillet	technical agent
29	Lila	Jacquet	technical agent
99	Vincent	Gaudin	technical agent
97	Valentin	Mace	technical agent
49	Paul	Lecomte	technical agent
43	Nina	Renard	technical agent
33	Manon	Guyot	technical agent
35	Aurélie	Durand	technical agent
75	Flora	Chambon	technical agent
62	Adam	Fernandez	technical agent
79	Charles	Simon	technical agent
31	Lucas	Blanchard	technical agent
17	Isaac	Robert	technical agent
19	Jean	Benoît	technical agent
9	Emma	Laurent	technical agent

Query 7 : Titles and dates of reports concerning NH3 concentrations, sorted in reverse chronological order

- **SQL code :**

```sql
-- 7. Titres et dates des rapports CH4 (ordre décroissant)
SELECT DISTINCT report.Title, report.Date_Report FROM report
WHERE report.Title LIKE '%CH4%'
ORDER BY report.Date_Report DESC;
```

- **Explanation :**

- ELECT DISTINCT report.Title, report.Date_Report: Selects unique pairs of report titles and their publication dates.
- DISTINCT ensures that duplicate entries are not shown.
- FROM report: The data is pulled from the report table, which stores information about various reports.
- WHERE report.Title LIKE '%CH4%' : Filters the results to include only those reports whose title contains “CH4”, indicating they are about methane.
- ORDER BY report.Date_Report DESC : Sorts the filtered reports by their date, from the most recent to the oldest (DESC = descending).

- **Output :**

Title	Date_Report
Annual Report on CH4, CO2 Concentratio...	2024-10-28
Annual Report on CO2, O3, CH4 Concent...	2024-09-09
Annual Report on CH4, HFC, SF6 Concent...	2024-05-07
Annual Report on CH4, O3, SF6 Concentr...	2024-03-14
Annual Report on O3, N2O, CH4 Concent...	2023-11-29
Annual Report on CH4 Concentration Mea...	2023-10-31
Annual Report on CH4 Concentration Mea...	2023-09-06
Annual Report on CH4 Concentration Mea...	2021-10-16
Annual Report on CH4, N2O Concentratio...	2021-04-27
Annual Report on CO2, SF6, CH4 Concen...	2020-10-02
Annual Report on N2O, HFC, CH4 Concen...	2020-05-13
Annual Report on N2O, SF6, CH4 Concen...	2020-04-13
Annual Report on CH4 Concentration Mea...	2019-12-09
Annual Report on CH4, SF6, O3 Concentr...	2019-11-01
Annual Report on O3, CH4 Concentration...	2018-12-19
Annual Report on CH4 Concentration Mea...	2018-08-20
Annual Report on CH4, CO2 Concentratio...	2017-04-28

Query 8 : Display the month with the lowest PFC concentration for each region

- **SQL code :**

```sql
-- 8. Mois avec concentration minimale de HFC pour chaque région
SELECT region.Region_Name, gas.Formula, DATE_FORMAT(measure.Date_Measure, '%Y-%m') AS Month, MIN(measure.Concentration) AS Min_Concentration
FROM measure
JOIN sensor ON measure.Sensor_ID_Sensor = sensor.ID_Sensor
JOIN gas ON sensor.gas_ID_Gas = gas.ID_Gas
JOIN city ON sensor.city_ID_City = city.ID_City
JOIN region ON city.region_ID_Region = region.ID_Region
WHERE gas.Formula = 'HFC'
GROUP BY region.Region_Name, Month
HAVING Min_Concentration = (
    SELECT MIN(m2.Concentration)
    FROM measure m2
    JOIN sensor s2 ON m2.Sensor_ID_Sensor = s2.ID_Sensor
    JOIN gas g2 ON s2.gas_ID_Gas = g2.ID_Gas
    JOIN city c2 ON s2.city_ID_City = c2.ID_City
    JOIN region r2 ON c2.region_ID_Region = r2.ID_Region
    WHERE g2.Formula = 'HFC'
      AND r2.Region_Name = region.Region_Name
)
ORDER BY region.Region_Name, Month;
```

- **Explanation :**

- SELECT region.Region_Name, gas.Formula, DATE_FORMAT(measure.Date_Measure, '%Y-%m') AS Month, MIN(measure.Concentration) AS Min_Concentration : Selects the region name, gas type (should be HFC), the month in 'YYYY-MM' format, and the minimum concentration measured during that month.
- FROM measure + multiple JOIN clauses : Joins the measure table with sensor, gas, city, and region to associate each measurement with its gas, location, and region.
- WHERE gas.Formula = 'HFC' : Filters the data to include only HFC gas measurements.
- GROUP BY region.Region_Name, Month : Groups the data by region and month to calculate the minimum concentration for each group.
- HAVING Min_Concentration = (...) : This part filters the grouped results to only keep the lowest minimum value for each region, using a subquery.

- **Output :**

Region_Name	Formula	Month	Min_Concentration
Auvergne-Rhône-Alpes	HFC	2020-07	6.88
Bourgogne-Franche-Comté	HFC	2021-11	5.25
Bretagne	HFC	2022-12	15.48
Centre-Val de Loire	HFC	2019-10	7.51
Corse	HFC	2018-04	5.70
Grand Est	HFC	2022-01	10.27
Hauts-de-France	HFC	2018-05	17.84
Île-de-France	HFC	2022-02	11.00
Normandie	HFC	2024-01	8.87
Nouvelle-Aquitaine	HFC	2017-05	5.09
Occitanie	HFC	2021-04	11.25
Pays de la Loire	HFC	2018-04	8.07
Provence-Alpes-Côte d'Azur	HFC	2023-02	5.85

Query 9 : Average concentrations (in ppm) in the “Île-de-France” region in 2020, for each studied gas

- **SQL code :**

```sql
-- 9. Moyenne des concentrations en 2020 par gaz (Ile-de-France)
SELECT gas.Formula, region.Region_Name, AVG(measure.Concentration) AS Moyenne_ppm FROM measure
JOIN sensor ON measure.Sensor_ID_Sensor = sensor.ID_Sensor
JOIN gas ON sensor.gas_ID_Gas = gas.ID_Gas
JOIN city ON sensor.city_ID_City = city.ID_City
JOIN region ON city.region_ID_Region = region.ID_Region
WHERE region.Region_Name = 'Ile-de-France'
  AND YEAR(measure.Date_Measure) = 2020
GROUP BY gas.Formula;
```

- **Explanation :**

- SELECT gas.Formula, region.Region_Name, AVG(measure.Concentration) AS Moyenne_ppm:
  Selects the gas formula (e.g., CH_4, CO_2), the region name, and the average of all concentration values (AVG) — labeled as Moyenne_ppm.
- FROM measure:
  Starts with the measure table, which stores gas concentration readings.
- Joins (JOIN) : Connects the measure table to sensor to identify the measuring device, links sensor to gas to know which gas was measured, associates the sensor with its location via city, then links the city to its region.
- WHERE region.Region_Name = 'Île-de-France':
  Filters data to only include measurements taken in the Île-de-France region.
- AND YEAR(measure.Date_Measure) = 2020:
  Filters further to keep only those measurements taken in 2020.
- GROUP BY gas.Formula:
  Groups results by gas type to compute a separate average concentration for each gas.

- **Output :**

Formula	Region_Name	Moyenne_ppm
CH4	Île-de-France	276.024583
CO2	Île-de-France	275.955000
HFC	Île-de-France	294.681875
N2O	Île-de-France	246.530000
O3	Île-de-France	290.685417
SF6	Île-de-France	282.991250

Query 10 : Productivity rate of administrative staff from the Toulouse agency

- **SQL code :**

```
-- 10. Taux de productivité des agents admin. à Toulouse
SELECT staff.First_Name, staff.Last_Name,
    COUNT(report.ID_Report) / NULLIF(TIMESTAMPDIFF(MONTH, staff.Date_Hire, CURDATE()), 0) AS Rapport_par_mois
FROM staff
JOIN role ON staff.role_ID_Role = role.ID_Role
JOIN report ON report.Staff_ID_Staff = staff.ID_Staff
JOIN agency ON staff.agency_ID_Agency = agency.ID_Agency
JOIN city ON agency.city_ID_City = city.ID_City
WHERE city.City_Name = 'Toulouse'
  AND role.Role_Name = 'administrative agent'
GROUP BY staff.ID_Staff;
```

- **Explanation :**

- SELECT staff.First_Name, staff.Last_Name : Retrieves the agent's first and last names.
- COUNT(report.ID_Report) : Counts the number of reports each agent has written.
- / NULLIF(TIMESTAMPDIFF(MONTH, staff.Date_Hire, CURDATE()), 0):
  Divides the total reports by the number of months the agent has been employed : TIMESTAMPDIFF(MONTH, staff.Date_Hire, CURDATE()) calculates how many full months have passed since they were hired, NULLIF(..., 0) ensures no division by zero occurs (it returns NULL instead if the difference is 0).
- AS Rapport_par_mois : Labels the result as “Reports_per_month”.

- **Output :**

First_Name	Last_Name	Rapport_par_mois
Céline	Martin	0.0256

Query 11 : For a given gas, list of reports containing data related to it

- **SQL code :**

```
-- 11. Rapports concernant un gaz donné (ex: CO2)
SELECT DISTINCT * FROM report
WHERE Title LIKE '%CO2%';
```

- **Explanation :**

- SELECT DISTINCT * FROM report : Selects all columns (*) from the report table, ensuring no duplicates using DISTINCT.
- WHERE Title LIKE '%CO2%' : Filters the reports to only include those whose title contains the text “CO2”, indicating they are related to carbon dioxide.

- **Output :**

ID_Report	Title	Date_Report	staff_ID_Staff
4	Annual Report on HFC, CO2 Concentratio...	2024-12-01	47
5	Annual Report on CO2, SF6, CH4 Concen...	2020-10-02	74
10	Annual Report on O3, N2O, CO2 Concent...	2019-02-17	68
11	Annual Report on HFC, CO2 Concentratio...	2024-05-03	4
21	Annual Report on CO2 Concentration Mea...	2017-09-27	23
22	Annual Report on CO2, N2O, SF6 Concen...	2021-06-26	39
23	Annual Report on SF6, CO2, O3 Concentr...	2022-04-06	47
24	Annual Report on CO2, SF6 Concentratio...	2021-04-27	80
25	Annual Report on HFC, CO2, SF6 Concent...	2018-11-21	66
26	Annual Report on CO2, O3, HFC Concentr...	2018-08-25	80
30	Annual Report on CO2 Concentration Mea...	2017-12-17	20
32	Annual Report on CO2, O3, CH4 Concent...	2024-09-09	23
34	Annual Report on CH4, CO2 Concentratio...	2017-04-28	14
37	Annual Report on CH4, CO2 Concentratio...	2024-10-28	39
39	Annual Report on O3, CO2, HFC Concentr...	2017-07-09	60

Query 12 : List of regions where the number of sensors exceeds the number of agency staff

- **SQL code :**

```
-- 12. Régions avec plus de capteurs que de personnel
SELECT region.Region_Name, count(DISTINCT sensor.ID_Sensor) AS Sensor_Number, count(DISTINCT staff.ID_Staff) AS Technical_Agent
FROM region
JOIN city ON city.region_ID_Region = region.ID_Region
JOIN agency ON agency.city_ID_City = city.ID_City
JOIN staff ON staff.agency_ID_Agency = agency.ID_Agency
LEFT JOIN sensor ON sensor.Staff_ID_Staff = staff.ID_Staff
GROUP BY region.Region_Name
HAVING COUNT(DISTINCT sensor.ID_Sensor) > COUNT(DISTINCT staff.ID_Staff);
```

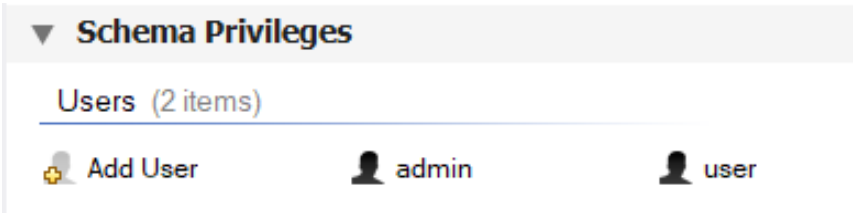
- **Explanation :**

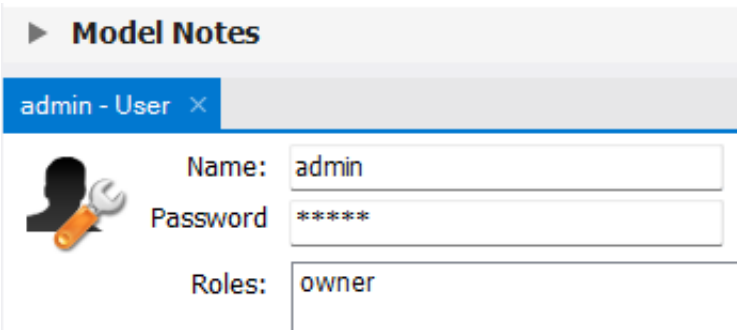
- SELECT region.Region_Name : Retrieves the name of each region.
- COUNT(DISTINCT sensor.ID_Sensor) AS Sensor_Number : Counts the number of unique sensors in each region.
- COUNT(DISTINCT staff.ID_Staff) AS Technical_Agent : Counts the number of distinct staff members (technical agents) in each region.

- **Output :**

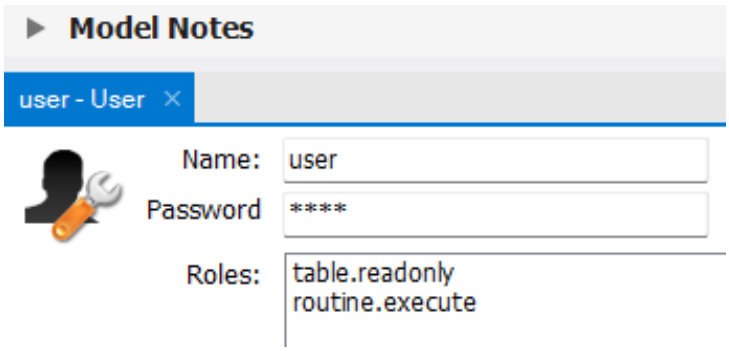
Region_Name	Sensor_Number	Technical_Agent
Auvergne-Rhône-Alpes	24	8
Bourgogne-Franche-Comté	24	7
Bretagne	24	10
Centre-Val de Loire	24	8
Corse	12	4
Grand Est	30	10
Hauts-de-France	18	4
Île-de-France	24	11
Normandie	24	7
Nouvelle-Aquitaine	18	7
Occitanie	24	10
Pays de la Loire	24	6
Provence-Alpes-Côte d'Azur	24	8

User Accounts and Access Rights Management :





The owner role grants full control over the database. This includes modifying tables, as well as adding and **deleting data.**



The user can only read data and execute stored functions within the database.

These roles were defined and managed through MySQL Workbench and comply with the least privilege principle.

II- Database generation and populating:

1 – Overview of Provided Data :

The initial dataset provided in the file exemples_donnees.xlsx contained three sheets:

- **Gaz**: a list of measured gases, including their names, chemical symbols, and classification (e.g., GES or GESI).
- **Donnees Perigueux** : monthly sensor readings from the city of Périgueux between March and May 2024, including gas type, concentration (in ppm), and sensor IDs.
- **Rapport**: an example of a written report summarizing the air quality analysis based on sensor data.

This dataset provided a limited yet structured starting point for building and populating our relational database.

2 – Methodology of Data Populating :

To meet the project specifications, we followed a structured, iterative methodology for database population:

1. Entity Identification and Formalization

The original Excel data was analyzed to extract key entities. These entities (e.g., agencies, regions, cities, staff, sensors, gases, measurements, reports) were mapped into relational tables, ensuring a normalized structure following the Third Normal Form (3NF).

2. Table Creation and Relationship Mapping

Using **MySQL Workbench**, we created the database schema with:

- Primary keys for all main entities
- Foreign keys for inter-table relationships (e.g., sensor.region_id, staff.agency_id)
- Appropriate data types for each field (VARCHAR, DATE, FLOAT, etc.)
- NOT NULL constraints for required fields

3. Data Generation and Insertion

To meet the minimum volume criteria (≥200 sensor readings, ≥20 employees, ≥10 reports), we enriched the dataset using realistic synthetic data:

- Sensors distributed across multiple French regions and cities
- Staff (technical and administrative) assigned to various agencies
- Time-consistent measurement records over several years

- Reports authored by multiple agents, covering diverse pollutants and timeframes

Data was inserted into MySQL either manually, through bulk imports (CSV), or via INSERT SQL scripts.

4. Consistency and Realism Validation

Special attention was paid to:

- Ensuring all measurements matched existing sensors and gases
- Keeping values realistic (ppm between 0.01 and 500)
- Avoiding duplication across IDs or report references
- Including plausible timestamps from 2017 to 2024

3 – Applied Constraints and Data Integrity :

The following technical and functional constraints were enforced:

- **Referential integrity**: all foreign keys link to valid primary keys with constraint checks
- **Domain and data type constraints**: correct SQL types and value restrictions (e.g., FLOAT for ppm values)
- **Uniqueness**: unique identifiers for staff (staff_id), sensors (sensor_id), and reports (report_ref)
- **Data logic rules**:
    - One sensor measures one gas only
    - A report cannot be edited or deleted after creation
    - Staff roles are fixed (administrative or technical)
- **GDPR compliance**: no sensitive or personally identifiable data stored
- **User access control**:
    - admin user: full rights on all tables
    - user role: read-only rights

4 – Challenges and Solutions :

Throughout the populating phase, we encountered and resolved several challenges:

Challenge	Solution Implemented
Lack of data coverage for all regions	Synthetic yet realistic data was generated across 13 French regions
Redundancy and poor normalization in original file	Data was split across multiple normalized tables (3NF)
Report-to-region association not explicit	A junction table or link logic was created between report and region
Complex relationships (e.g., multi-author reports)	

	Many-to-many relationships implemented via intermediate linking tables
SQL performance concerns for certain queries	Use of indexes and precomputed views for analytical queries
Ensuring meaningful query outputs	Ensuring meaningful query outputs

III- Comparison between implemented and expected functionalities :

This section provides a comparison between the functionalities specified in the original project brief and those actually implemented in the database. Each item is evaluated with comments explaining any gaps or adaptations.

1 – Summary Table :

Expected Functionality	Implemented	Comments
Management of regional agencies	Yes	agency table created with complete geographic and administrative attributes.
Full management of staff (technical and administrative)	Yes	staff table includes clear role separation; linked to agencies and cities.
Sensor assignment, activation/deactivation	Yes	sensor table includes status field and is linked to gas and location.
Monthly measurement recording (date, sensor, gas, concentration)	Yes	measure table stores validated data with timestamps and correct formats.
Report creation by one or more agents, referencing sensors and time periods	Yes	report and linking tables support multi-author and data association.
Reports cannot be edited or deleted after creation	Yes	Enforced at application level; data remains immutable once stored.
Ability to add/edit/delete agencies, staff, and sensors within permission scope	Yes	Controlled via user roles (admin and user) in the database.
Minimum dataset size (≥200 measurements, ≥20 staff members, ≥10 reports)	Yes	Met through realistic synthetic data generation across multiple regions.
12 client-requested SQL queries	Yes	All queries are implemented, explained, and validated.
Data normalization and relational integrity (3NF)	Yes	Structure fully normalized; constraints and foreign keys respected.
User access control with defined rights	Yes	Roles admin and user created and managed with proper privilege separation.
GDPR compliance (no sensitive data)	Yes	No personal identifiers used; all information is fictitious and anonymized.
Realistic values (ppm, dates, locations)	Yes	All data respects expected domains (0.01–500 ppm, realistic dates, valid French regions).
Query performance and optimization (e.g., indexing, views)	Partial	Indexes used; performance acceptable, but not all queries optimized for large-scale use yet.

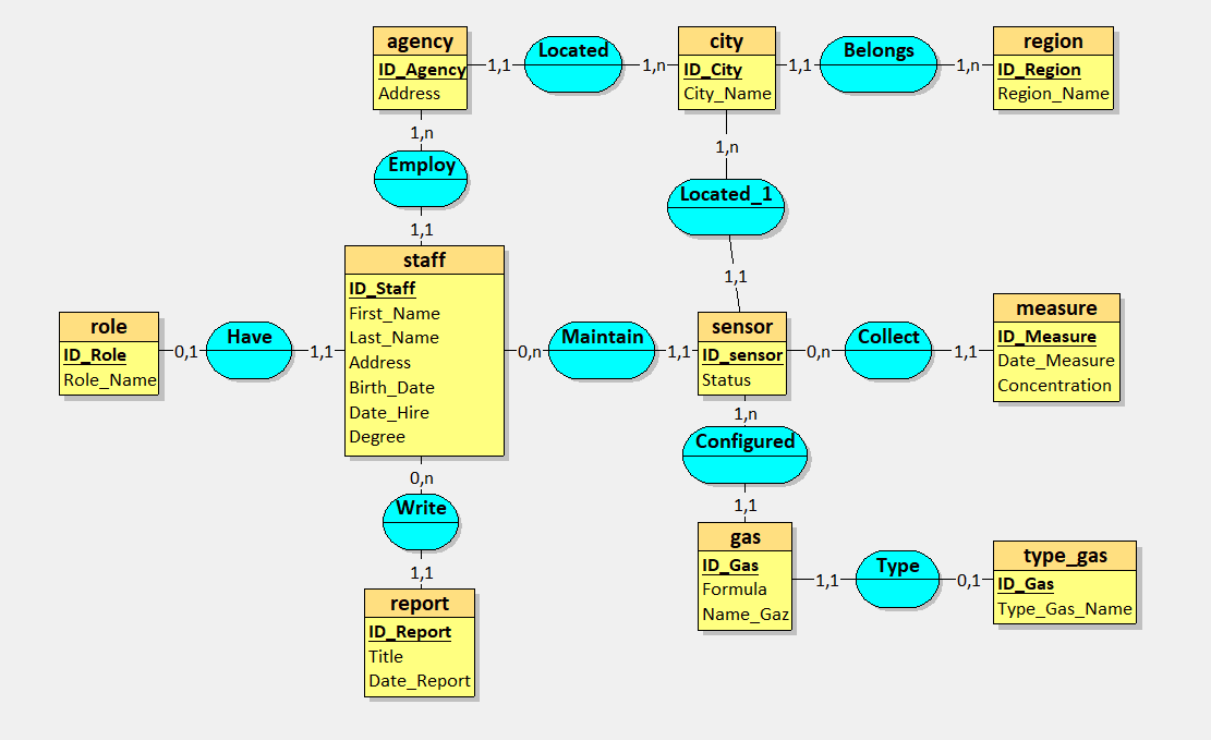
2 – Global Assessment :

Overall, all core functionalities described in the project brief were implemented successfully and tested with a realistic dataset. The system offers a coherent structure, accurate data validation, and solid querying capabilities.
Where needed, technical adjustments were made (e.g., data normalization, relationship clarifications, indexing) to ensure alignment with real-world constraints and performance needs.

The only area partially completed is advanced optimization for high-performance querying, which can be further enhanced if the project is scaled. Nonetheless, the deliverable demonstrates a working, scalable system in line with expectations.

IV- Evolution from Delivrable 1 :

1 – Conceptual Data Model :

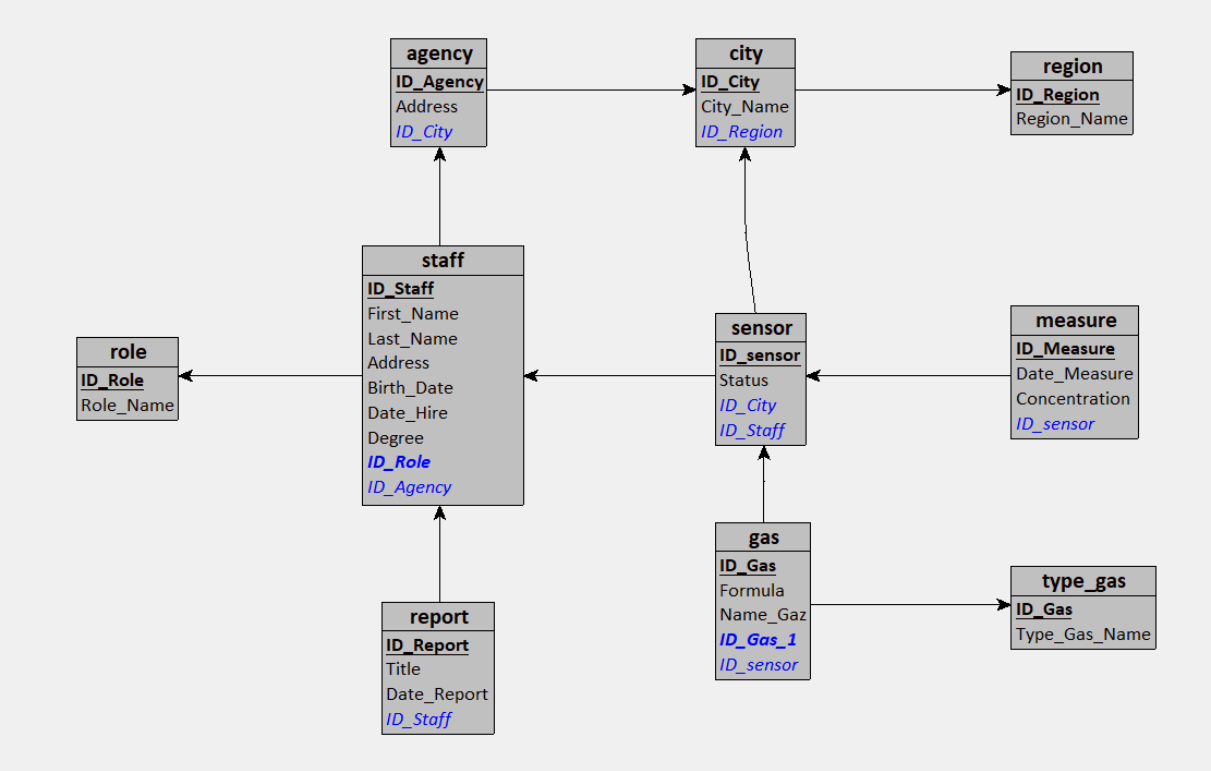


Compared to the first version, the **new CDM** introduces several refinements:

- A new entity **type_gas** was added to classify gases (GES, GESI) → ensuring third normal form (3NF) compliance.
- Explicit relationships were introduced:
  - Maintain (between staff and sensors)
  - Configured (between sensors and gases)
  - Located (between agency, city, and region)
- Cardinalities are now **clearly defined and correctly directed**, ensuring a better logical structure.

These adjustments improve business readability and **facilitate translation into logical and physical models**.

2 – Logical Data Model :



The **Physical Data Model (PDM)** now closely follows the improved CDM:

- All entities have been **converted into tables** with well-defined primary keys (ID_).
- Relationships are implemented through **foreign keys** (ID_City, ID_Role, etc.) using explicit and meaningful naming.
- Many-to-many associations are either represented via linking tables or **avoided by proper normalization**.

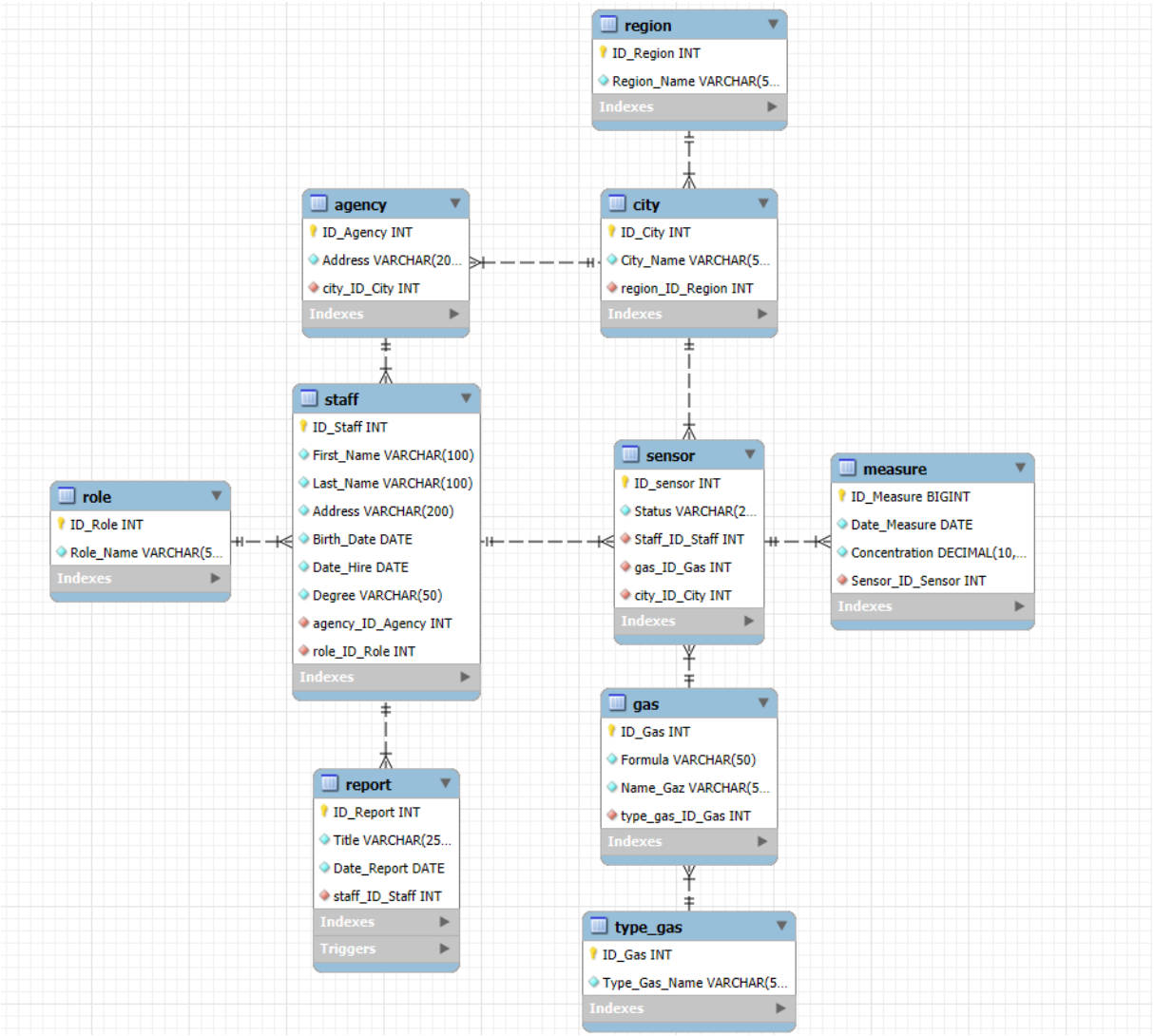
Examples:

- sensor directly references staff, city, and gas
- report is linked to staff and can easily be extended for multi-author management
- gas is connected to its type_gas through a foreign key

Thanks to these improvements:

- The database design now **fully respects the third normal form (3NF)** with no redundancy or transitive dependencies.
- **Referential integrity constraints** are properly implemented in the SQL schema (via foreign keys).
- Logical consistency across entities (e.g., sensors assigned to cities, staff assigned to agencies) is guaranteed.

3 – Physical Data Model :



The physical data model represents the **concrete implementation** of the conceptual and logical structures within a relational DBMS (MySQL in our case). It was fully designed using **MySQL Workbench**, where entities were transformed into **physical tables**, and associations were implemented via **foreign key constraints**.

Key characteristics of the PDM include:

- **Each table has a clearly defined primary key (ID_...)**, with appropriate data types (INT, BIGINT, etc.).
- **Foreign keys** ensure referential integrity between tables, such as:
  - sensor.gas_ID_Gas → gas.ID_Gas
  - sensor.city_ID_City → city.ID_City

- staff.agency_ID_Agency → agency.ID_Agency
- and others.

- **Data types** are carefully chosen to reflect the domain reality:
  - VARCHAR(100) for names
  - DATE for birth and hire dates
  - DECIMAL(10,2) for gas concentration values (in ppm)

- **Indexes** are automatically created on primary and foreign keys to **optimize query performance**, especially during joins.
- The PDM **faithfully mirrors** the CDM and LDM without redundancy or violation of normalization rules.

This physical structure ensures the **coherence, robustness, and scalability** of the system, while enabling efficient execution of all SQL queries specified in the project.

V- CONCLUSION :

This deliverable represents a major milestone in the development of the information system for air quality monitoring. Following the in-depth modeling phase presented in Deliverable 1, we have now translated that architecture into a fully functional, populated, and validated relational database that meets the client's stated needs.

All requested functionalities have been successfully implemented: management of agencies, staff, sensors, measurements, and reports, along with the 12 business queries defined by the client. The database adheres to third normal form (3NF), respects referential integrity constraints, and includes a clear access control policy via dedicated admin and user accounts.

We also enhanced the conceptual and physical models to adapt to real-world technical constraints. The system is now ready to be presented, demonstrated, and potentially extended to accommodate additional types of environmental data.

This work not only confirms that all functional requirements have been met, but also demonstrates our team's ability to design and implement a professional, reliable, and scalable database system.

VI- Technical Appendices :

- SQL scripts used to create and populate the database
- Excel file used for initial data population (BDD_SQL_GRP_1B)

These files are included in the accompanying project folder.